

STAGE 1

Messaging MVP and Human-Gated Agent Workflow

A delivery charter for a secure, responsive, one-to-one messaging application.

VERSION	DATE	STATUS
1.0	2026-07-28	Project foundation

Human approval remains the gate for design readiness, task creation, pull-request approval, and final merge.

Revision history

Version	Date	Status	Summary
1.0	2026-07-28	Project foundation	Established the Stage 1 MVP boundary, architecture, delivery workflow, quality gates, and maintenance rules.

1. Summary and MVP Boundary

Create a responsive web messaging application in which two registered users can find each other, start a private conversation, and exchange persistent text messages in real time.

Stage 1 is complete when every MVP story has passed automated tests, been reviewed by the user, and been squash-merged by a Claude Code delivery agent. Production deployment is deferred.

Included features

- Email/password registration, sign-in, session persistence, and logout.
- Unique username for discovering another registered user.
- Username search and creation of one-to-one conversations.
- Conversation list showing participant, latest message, and timestamp.
- Paginated message history.
- Sending and receiving plain-text messages in real time.
- Pending, failed, empty, loading, and disconnected UI states.
- Responsive desktop and mobile-browser layouts.
- Basic keyboard accessibility and visible focus states.
- Authorization preventing access to conversations belonging to other users.

Explicitly deferred

- Group conversations, attachments, reactions, edit/delete, read receipts, typing indicators, presence, message search, push notifications, calls, blocking, moderation, end-to-end encryption, offline/PWA support, and production deployment.

2. Technology and Interface Design

Recommended stack

Area	Stage 1 choice	Purpose
Languages	TypeScript and SQL	One primary language across client, API, agents, and tests
Runtime/package management	Current Node.js LTS, pnpm workspaces, Turborepo	Reproducible monorepo with shared scripts and caching
Frontend	Next.js App Router, React, Tailwind CSS, shadcn/ui	Responsive interface with accessible components
Client data	TanStack Query, React Hook Form, Zod	API caching, forms, and client validation without Redux
Backend	NestJS with its standard Express adapter	Structured modules, guards, validation, and testable services
Realtime	NestJS WebSocket gateway and Socket.IO	Authenticated rooms and realtime message notifications
Database	PostgreSQL and Prisma ORM/Migrate	Relational storage, type-safe queries, and committed migrations
Sessions/security	express-session, PostgreSQL session store, Argon2id, Helmet, Nest throttling	Revocable server-side sessions and hardened authentication
API contract	REST, OpenAPI via @nestjs/swagger, generated TypeScript types	One documented contract shared with the web client
Unit/component tests	Vitest and React Testing Library	Fast tests for business logic and UI states
API/integration tests	Nest testing utilities, Supertest, Testcontainers	Real PostgreSQL and authorization testing
Browser tests	Playwright and axe accessibility checks	Multi-user flows, responsive UI, and accessibility
Local environment	Docker Compose	Consistent PostgreSQL and full-stack startup
CI	GitHub Actions	Type checking, linting, tests, build, and migration validation
Design/tracking/version control	Figma, Jira Cloud, GitHub	Design source, work source of truth, and code source of truth

Use stable versions available at repository initialization and commit the lockfile; dependency upgrades occur through separate Jira items.

Architecture and public interfaces

- Monorepo areas: apps/web, apps/api, shared generated API types, and end-to-end tests.

- The web application proxies HTTP and Socket.IO traffic to the API so browser authentication remains same-origin.

- Passwords accept 15-128 characters and are stored only as Argon2id hashes.

Sessions use PostgreSQL-backed opaque cookies with HttpOnly, Secure in HTTPS environments, and explicit SameSite protection.

- Core entities: User, Conversation, ConversationMember, Message, and the session store.

- Direct conversations have a canonical participant-pair key with a uniqueness constraint, preventing duplicate conversations.

- Messages contain a client-generated idempotency key, sender, conversation, body, and server timestamp. Bodies are trimmed and limited to 2,000 characters.

- REST is the only message-write path; Socket.IO broadcasts committed results.

This avoids different persistence behavior between HTTP and realtime paths.

Minimum endpoints:

- POST /auth/register, POST /auth/login, POST /auth/logout, GET /auth/me
- GET /users?query= for username-prefix lookup
- GET /conversations, POST /conversations/direct
- GET /conversations/:id/messages?cursor=&limit=
- POST /conversations/:id/messages
- Consistent error envelope: { code, message, fieldErrors? }

Realtime events:

- conversation.created informs both participants about a new direct conversation.
- message.created broadcasts a committed message to authorized conversation members.
- Socket connections authenticate from the server-side session, validate an explicit origin allowlist, enforce membership, limit payload size, and close when the session expires or the user logs out.

The choices align with the official [NestJS WebSocket model](#), [Prisma/NestJS integration](#), and [OWASP session, password, and WebSocket guidance](#).

3. Agents, Services, and Guardrails

All development agents are custom Claude Code agents. The user invokes them individually; no coordinator agent, Jira trigger, unattended daemon, or event-driven orchestration is introduced.

Agent/service	Responsibility	Permissions and limits
Figma Design Agent	Draft responsive auth, conversation-list, chat, and state frames through Figma MCP	Figma write access; no Jira creation or code merge
Issue Analyst Agent	Read an existing Jira item, Figma frames, and repository; verify Definition of Ready and prepare an implementation note	Read-only for code; may comment or transition Jira but cannot create the final task
Implementer Agent	Write production code, migrations, and required tests using Claude Code	Feature-branch writes only; cannot push to main, approve, or merge
QA Agent	Run unit, integration, browser, accessibility, and security scenarios; add test-only changes	May edit tests; product-code failures return to the Implementer
Review Agent	Independently inspect the diff for correctness, authorization, regressions, unnecessary scope, and acceptance-criteria coverage	Read-only; publishes blocking/non-blocking findings
Delivery Agent	Create branches, commit/push approved work, open/update PRs, update Jira, and perform the final merge	Cannot edit product code or bypass protection; merges only after explicit user approval
Jira Cloud	Source of truth for scope, acceptance criteria, ownership, status, and decisions	User personally reviews and creates every work item
Figma	Source of truth for responsive UI and state designs	User must approve frames before related work becomes Ready
GitHub	Repository, PRs, reviews, checks, and audit history	Protected main; required checks and one human approval
GitHub Actions	Executes deterministic quality gates	No automatic merge
PostgreSQL/Docker	Local persistence and isolated integration environments	No production environment in Stage 1

Use Claude Code's custom-agent and MCP support with the official [Claude Code agent model](#), [Figma MCP server](#), and [Atlassian Roovo MCP server](#).

Required repository protection

- Protect main; prohibit direct pushes, force pushes, and branch-protection bypass.
- Require all CI checks, resolved conversations, and one approval from the user.
- Agents cannot approve their own PRs.
- Use squash merge and automatically delete merged branches.
- No secrets in repository files, prompts, logs, screenshots, Jira comments, or Figma annotations.
- The Delivery Agent must stop if approval, CI, Jira linkage, or required review evidence is missing.

4. Step-by-Step Workflow

One-time setup

1. Create the GitHub repository and enable protected-branch rules.
2. Create a Jira project with Backlog, Ready, In Progress, In Review, and Done.
3. Create the Stage 1 Jira epic.
4. Create a Figma file with Foundations, Flows, and Responsive Screens pages.
5. Connect Claude Code to the official Figma and Atlassian MCP servers and authenticate GitHub CLI with least-privilege access.
6. Add the six Claude Code agent definitions and prohibit delegation between them.
7. Establish the monorepo, Docker Compose environment, CI checks, issue template, and PR template through an initial enablement ticket.
8. Create one independently mergeable Jira story per vertical slice:
 - Registration, sign-in, session, and logout.
 - User discovery and direct-conversation creation.
 - Conversation list and persisted message history.
 - Sending messages and idempotent persistence.
 - Realtime delivery, reconnection, and logout invalidation.
 - Responsive states, accessibility, and final security hardening.

Design and task preparation

1. Invoke the Figma Design Agent with the Stage 1 feature boundary.
2. Have it draft desktop and mobile-width frames for auth, conversation list, active chat, loading, empty, error, and disconnected states.
3. Personally review and approve or revise the designs.
4. Ask the Issue Analyst Agent to draft each Jira story with user value, scope, non-goals, acceptance criteria, Figma links, API/data impact, dependencies, and test expectations.
5. Personally edit and create the Jira items. Only the user may mark a task Ready.

Repeat for every Jira story

1. Select one Ready issue and manually invoke the Issue Analyst Agent with its key.
2. The agent confirms the issue has approved Figma context, testable acceptance criteria, bounded scope, dependencies, and one-PR feasibility. It comments and stops if anything is missing.
3. Invoke the Delivery Agent to create type/JIRA-KEY-short-description, link the branch, and transition Jira to In Progress.
4. Invoke the Implementer Agent. It reads Jira and Figma, implements only the

stated vertical slice, adds migrations and tests, and records decisions in the PR-ready summary.

5. Invoke the QA Agent. It runs the complete affected test set and verifies the acceptance criteria. Test failures return to the Implementer; repeat until clean.

6. Invoke the Review Agent. It performs an independent read-only review, emphasizing authorization, session handling, WebSocket membership, data isolation, error states, and scope creep.

7. Resolve blocking findings through another Implementer -> QA -> Review loop.

8. Invoke the Delivery Agent to push the branch, open a PR, link Jira and Figma, summarize the change and evidence, and transition Jira to In Review.

9. GitHub Actions runs all required gates. The agent may diagnose failures, but code corrections return to the Implementer.

10. Personally inspect code quality and either request changes or approve the PR.

11. For requested changes, repeat the Implementer -> QA -> Review -> Delivery cycle.

12. After approval, explicitly instruct the Delivery Agent to merge.

13. The Delivery Agent rechecks approval, required CI, unresolved discussions, and current branch state; squash-merges without bypassing protection.

14. The Delivery Agent updates Jira to Done and adds the PR, merge commit, test summary, and any follow-up items.

15. Handle follow-up work through new user-created Jira items rather than silently expanding the completed story.

5. Test Plan, Acceptance, and Plan Maintenance

Mandatory automated scenarios

- Register, sign in, restore a session, log out, and reject expired or invalid sessions.
- Reject duplicate email/username without leaking account details.
- Prevent unauthenticated access and cross-user access to conversations or messages.
- Prevent unauthorized Socket.IO origins, missing sessions, and non-member room access.
- Start only one direct conversation for the same user pair.
- Send a message once when the same idempotency key is retried.
- Exchange messages between two isolated browser contexts and retain them after reload.
- Paginate message history without gaps or duplicates.
- Show pending, failed, disconnected, empty, and reconnecting states.
- Operate at representative desktop and mobile widths with keyboard-accessible controls.
- Apply migrations to a clean PostgreSQL database and build both applications successfully.

Every PR must pass formatting, linting, strict type checking, unit/component tests, API integration tests, browser tests for affected flows, migration validation, and production builds. Playwright supplies browser isolation, traces, and cross-browser support as documented in its [official testing guide](#).

Stage 1 completion criteria

- All listed MVP behaviors are merged through protected PRs.
- Every Jira story links to one merged PR and approved Figma frames.
- No critical/high security findings or unresolved blocking review comments remain.
- Two users can complete the full direct-message flow on desktop and mobile browser widths.
- Deferred features remain absent from the implementation and backlog unless the user creates new work.

Updateable plan artifacts

Maintain two synchronized deliverables:

- docs/plans/Stage-1-Messaging-App-Plan-v1.0.md as the editable source.
- output/pdf/Stage-1-Messaging-App-Plan-v1.0.pdf as the polished distribution copy.

Future refinements update the Markdown source, append the revision history, increment the version, regenerate the PDF, render every PDF page to images for visual inspection, and verify its text and page count before release.

Assumptions

- Stage 1 targets a responsive web application and one-to-one messaging only.
- The custom backend runs on Node.js with local PostgreSQL through Docker

Compose.

- Figma designs are agent-drafted but human-approved.
- The user personally creates Jira items and is the required GitHub reviewer.
- Agents perform all implementation, validation, PR, Jira-transition, and merge

operations after task creation.

- The user manually invokes every agent and gives the explicit merge command;

this is the Stage 1 substitute for an orchestrator.

- Hosting, observability infrastructure, automated deployments, Jira-triggered

agents, and Claude Agent for Jira are candidates for later stages.

6. Decision Register

Decision	Stage 1 position
Product boundary	Responsive web, direct messaging only
Persistence	PostgreSQL through Prisma
Authentication	Revocable PostgreSQL-backed server sessions
Message write path	REST only; Socket.IO broadcasts committed results
Conversation uniqueness	Canonical participant-pair key with a database uniqueness constraint
Delivery control	Human-reviewed, protected, squash-merged pull requests
Agent orchestration	Manual invocation only; no unattended coordinator
Deployment	Deferred beyond Stage 1

7. Official References

Reference	Official URL
NestJS WebSocket gateways	https://docs.nestjs.com/websockets/gateways
Prisma with NestJS	https://docs.prisma.io/docs/guides/frameworks/nestjs
OWASP WebSocket Security Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/WebSocket_Security_Cheat_Sheet.html
OWASP Session Management Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
OWASP Password Storage Cheat Sheet	https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html
Playwright testing	https://playwright.dev/docs/intro
Claude Code subagents	https://code.claude.com/docs/en/sub-agents
Figma MCP server	https://help.figma.com/hc/en-us/articles/32132100833559-Guide-to-the-Figma-MCP-server
Atlassian Rovo MCP server	https://developer.atlassian.com/cloud/rovo-mcp/